

TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 8: Dynamic Programming I: Weighted Interval Scheduling and Knapsack

Foreign Trade University

Contents

1	Introduction: The Dynamic Programming Paradigm	1
1.1	When does dynamic programming apply?	1
1.2	Two implementation styles: memoization vs. bottom-up	2
1.3	A recipe for designing a DP	2
2	Weighted Interval Scheduling	2
2.1	Problem statement	2
2.2	Why greedy fails, and what replaces it	3
2.3	The recurrence	3
2.4	Algorithm and running time	3
2.5	Reconstructing an optimal solution	4
3	Subset Sum and the 0/1 Knapsack Problem	4
3.1	Subset Sum	4
3.2	0/1 Knapsack	4
3.3	Why $O(nW)$ is only <i>pseudo</i> -polynomial	5
4	Unbounded Knapsack: Coin Change and Rod Cutting	5
4.1	Coin change: minimum number of coins	6
4.2	Coin change: number of ways	6
4.3	Rod cutting	6
5	Longest Increasing Subsequence	6
6	Summary and Looking Ahead	7

1 Introduction: The Dynamic Programming Paradigm

This week we begin our study of **dynamic programming** (DP), arguably the most powerful general-purpose algorithm-design technique in this course. Where greedy algorithms (Weeks 4–5) commit to a single locally-optimal choice at each step and never look back, dynamic programming instead *considers all the choices available at each step*, but does so efficiently by reusing the answers to subproblems it has already solved.

(Note: this is the Midterm week – the material below also serves as a consolidation of the problem-solving habits developed so far: define the object precisely, write a recurrence, prove it correct, analyze its cost, and verify empirically.)

The name “dynamic programming” is historical (Richard Bellman, 1950s; “programming” here means “planning,” not writing code). The essential idea is captured by one slogan:

Solve each subproblem once, store its answer, and build the answer to a larger problem from the answers to smaller ones.

1.1 When does dynamic programming apply?

Two structural properties must hold for a problem to admit an efficient DP solution.

Definition 1.1 (Optimal substructure). *A problem has optimal substructure if an optimal solution to the whole problem can be expressed in terms of optimal solutions to its subproblems. Equivalently: once we fix one “decision” (e.g. whether to include the last item), what remains is an independent, smaller instance of the same problem, and the overall optimum uses an optimum of that smaller instance.*

Definition 1.2 (Overlapping subproblems). *A problem has overlapping subproblems if a naive recursive solution would solve the same subproblem many times. This is what distinguishes dynamic programming from plain divide-and-conquer (where subproblems are typically disjoint, as in merge sort).*

When both hold, we can write a **recurrence** (the Bellman equation) expressing the optimum of an instance in terms of the optima of smaller instances, and then evaluate it without recomputation. The textbook example of overlapping subproblems is the Fibonacci recurrence $F(n) = F(n-1) + F(n-2)$: the naive recursion makes $2F(n+1) - 1$ calls (exponentially many), whereas remembering each $F(k)$ reduces this to $n + 1$ distinct subproblems. Problem 12 asks you to instrument exactly this blow-up and confirm that DP does linear work.

1.2 Two implementation styles: memoization vs. bottom-up

There are two equivalent ways to evaluate a DP recurrence without recomputation.

- **Top-down (memoization).** Write the recurrence as a recursive function, but cache (“memoize”) the result of each subproblem the first time it is computed; on subsequent calls, return the cached value immediately. Subproblems are solved *lazily*: only those actually reached are ever computed. This is often the easiest to write (it mirrors the recurrence directly) and avoids computing unreachable subproblems.
- **Bottom-up (tabulation).** Identify an order in which subproblems depend only on *smaller* ones, then fill a table iteratively in that order, so that whenever we need a subproblem’s value it is already in the table. This avoids recursion overhead, makes the running time and space transparent, and often enables space optimizations (e.g. keeping only the last row of a 2-D table).

Both compute the same values and have the same asymptotic running time (number of distinct subproblems times the work per subproblem). Problem 4 asks you to implement a DP both ways and confirm they agree. We will generally present the bottom-up form for analysis and the top-down form when the recurrence is more natural recursively.

1.3 A recipe for designing a DP

For every DP this week (and in Week 9), we follow the same five steps:

- (1) **Define the subproblem** precisely, in words and as an array (e.g. “ $M[j]$ = optimal value over the first j intervals”).
- (2) **Write the recurrence** relating a subproblem to smaller ones, including the **decision** being made (take vs. skip, cut here vs. later).
- (3) **State the base cases.**
- (4) **Choose an evaluation order** (bottom-up) or memoize (top-down), and analyze the running time as (number of subproblems) $\times$ (work per subproblem).
- (5) **Reconstruct an optimal solution** by tracing back through the table, not just reporting the optimal *value*.

2 Weighted Interval Scheduling

2.1 Problem statement

Recall the (unweighted) Interval Scheduling problem from Week 4, where we maximized the *number* of compatible intervals and the earliest-finish-time greedy algorithm was optimal. We now attach a **value** to each interval.

Definition 2.1 (Weighted Interval Scheduling). *We are given n requests, where request i has a start time s_i , a finish time f_i (with $s_i < f_i$), and a weight (or value) $w_i \geq 0$. Two requests are compatible if their half-open intervals $[s_i, f_i)$ and $[s_j, f_j)$ do not overlap. The goal is to select a set of mutually compatible requests of **maximum total weight**.*

2.2 Why greedy fails, and what replaces it

The earliest-finish-time greedy rule maximizes the *count* of intervals, but a single high-value interval can be worth more than many low-value ones it conflicts with. Sorting by weight fails too: a very heavy interval might block two even-heavier compatible ones. No simple greedy ordering is correct – so we turn to dynamic programming.

Sort the requests by finish time, so $f_1 \leq f_2 \leq \dots \leq f_n$. The key quantity is:

Definition 2.2 (The function $p(j)$). *For each request j , let $p(j)$ be the largest index $i < j$ such that request i is compatible with request j (i.e. $f_i \leq s_j$), or 0 (“none”) if no such i exists.*

Because the requests are sorted by finish time, $p(j)$ can be found by **binary search** on the array of finish times: $p(j)$ is the rightmost interval whose finish time is $\leq s_j$. Computing all of $p(\cdot)$ thus takes $O(n \log n)$. Problem 1 asks you to implement this.

2.3 The recurrence

Consider the last interval, request n (the one finishing last). Any optimal solution either *includes* it or not – and this single binary decision is the heart of the DP.

- If the optimum **includes** n , then it can contain no interval that overlaps n ; the remaining intervals it may use are exactly $1, \dots, p(n)$. So the optimum is w_n plus the optimum over $\{1, \dots, p(n)\}$.
- If the optimum **excludes** n , then it is simply the optimum over $\{1, \dots, n-1\}$.

Let $M[j]$ denote the maximum total weight obtainable using only requests $1, \dots, j$ (after sorting by finish time). Then:

$$M[0] = 0, \quad M[j] = \max(M[j-1], w_j + M[p(j)]).$$

Theorem 2.3 (Correctness of the recurrence). *For every j , $M[j]$ as defined above equals the maximum total weight of a compatible subset of $\{1, \dots, j\}$.*

Proof. By strong induction on j . The base case $M[0] = 0$ is the empty instance. For $j \geq 1$, let O_j be an optimal compatible subset of $\{1, \dots, j\}$. Either $j \in O_j$ or not. If $j \notin O_j$, then O_j is a compatible subset of $\{1, \dots, j-1\}$, so its weight is at most $M[j-1]$ (by the inductive hypothesis), and conversely any compatible subset of $\{1, \dots, j-1\}$ is also valid for $\{1, \dots, j\}$, so this branch contributes exactly $M[j-1]$. If $j \in O_j$, then no interval in O_j overlaps j , so every other member has index $\leq p(j)$; thus $O_j \setminus \{j\}$ is a compatible subset of $\{1, \dots, p(j)\}$ of weight $M[j] - w_j$, which by induction is at most $M[p(j)]$, giving weight at most $w_j + M[p(j)]$; and this value is achievable (take an optimum of $\{1, \dots, p(j)\}$ plus j). The optimum is the better of the two branches, which is exactly the stated maximum. $\square$

2.4 Algorithm and running time

Algorithm 1 Weighted-Interval-Scheduling (bottom-up)

- 1: Sort requests by finish time so that $f_1 \leq \dots \leq f_n$
 - 2: Compute $p(1), \dots, p(n)$ by binary search on finish times
 - 3: $M[0] \leftarrow 0$
 - 4: **for** $j = 1$ to n **do**
 - 5: $M[j] \leftarrow \max(M[j-1], w_j + M[p(j)])$
 - 6: **end for**
 - 7: **return** $M[n]$
-

Sorting is $O(n \log n)$; computing all $p(j)$ is $O(n \log n)$; the main loop is $O(n)$. Total: $O(n \log n)$. Problems 2 (value) and 4 (top-down memoization) implement this.

2.5 Reconstructing an optimal solution

The array M gives the optimal *value*; to recover the optimal *set* we trace back. Starting at $j = n$, request j is in the optimum iff the “take” branch was at least as good as the “skip” branch:

$$w_j + M[p(j)] \geq M[j-1].$$

If so, output j and jump to $j \leftarrow p(j)$; otherwise move to $j \leftarrow j-1$. This runs in $O(n)$. Problem 3 asks you to implement this traceback and verify that the reconstructed set is feasible and has weight exactly $M[n]$.

3 Subset Sum and the 0/1 Knapsack Problem

3.1 Subset Sum

Definition 3.1 (Subset Sum). *Given n non-negative integers $a_1, \dots, a_n$ and a target T , decide whether some subset of the a_i sums to exactly T .*

Let $R[i][t]$ be true iff some subset of the first i numbers sums to t . Considering item i (take it or skip it):

$$R[0][0] = \text{true}, \quad R[0][t] = \text{false} \ (t > 0), \quad R[i][t] = R[i-1][t] \vee (t \geq a_i \wedge R[i-1][t - a_i]).$$

The answer is $R[n][T]$. The table has $(n+1)(T+1)$ Boolean entries, each filled in $O(1)$, so the running time is $O(nT)$. Problem 5 asks you to implement this, using the standard 1-D optimization: keep a single Boolean array **reachable** of length $T+1$ and, for each a_i , update it for t from T down to a_i . The descending order is essential – it ensures each item is used *at most once* (if we updated in ascending order, a_i could be reused, which would instead solve the *unbounded* version).

3.2 0/1 Knapsack

Definition 3.2 (0/1 Knapsack). *Given n items, where item i has an integer weight $w_i \geq 0$ and value $v_i \geq 0$, and a knapsack of integer capacity W , select a subset of items of maximum total value whose total weight is at most W . Each item may be taken at most once (hence “0/1”).*

Subset Sum is the special case $v_i = w_i$, $W = T$, asking only for feasibility. The DP is the natural generalization. Let $dp[i][w]$ be the maximum value achievable using the first i items with capacity w :

$$dp[0][w] = 0, \quad dp[i][w] = \begin{cases} dp[i-1][w] & \text{if } w_i > w, \\ \max(dp[i-1][w], v_i + dp[i-1][w - w_i]) & \text{otherwise.} \end{cases}$$

The first case says: if item i does not fit, we must skip it. The second case is the take-vs-skip decision. The answer is $dp[n][W]$.

Algorithm 2 Knapsack-0/1 (bottom-up, 2-D table)

```

1: for  $w = 0$  to  $W$  do
2:    $dp[0][w] \leftarrow 0$ 
3: end for
4: for  $i = 1$  to  $n$  do
5:   for  $w = 0$  to  $W$  do
6:      $dp[i][w] \leftarrow dp[i-1][w]$ 
7:     if  $w_i \leq w$  then
8:        $dp[i][w] \leftarrow \max(dp[i][w], v_i + dp[i-1][w - w_i])$ 
9:     end if
10:  end for
11: end for
12: return  $dp[n][W]$ 

```

Theorem 3.3. *Algorithm Knapsack-0/1 computes the optimal value in $O(nW)$ time and $O(nW)$ space.*

Proof. Correctness follows from the recurrence by induction on i , exactly as for Weighted Interval Scheduling: $dp[i][w]$ considers the binary decision on item i and defers the rest to optimal subinstances $dp[i-1][\cdot]$ (optimal substructure). The table has $(n+1)(W+1)$ entries, each computed in $O(1)$, giving $O(nW)$. $\square$

Problem 6 implements the value; Problem 7 reconstructs the chosen items by tracing back: item i was taken iff $dp[i][w] \neq dp[i-1][w]$, in which case subtract w_i from w and continue to row $i-1$. (A standard space optimization keeps only one row, but then reconstruction requires either storing decisions or a divide-and-conquer trick; we keep the full table for clarity.)

3.3 Why $O(nW)$ is only *pseudo-polynomial*

The running time $O(nW)$ looks polynomial, and indeed it is polynomial in the *numeric value* W . But the *size of the input* – the number of bits needed to write it down – is only $O(n \log W)$, because the capacity W is written in binary using about $\log_2 W$ bits. So W itself is *exponential* in the input size: doubling the number of bits of W squares W , and hence squares the running time. An algorithm whose running time is polynomial in the numeric values of the inputs, but exponential in their bit-length, is called **pseudo-polynomial**.

This matters because 0/1 Knapsack is **NP-hard**: no genuinely polynomial-time (in the input size) algorithm is known, and one is believed not to exist. The $O(nW)$ DP is efficient *only when W is small* (e.g. bounded by a polynomial in n). For instances with huge capacities, the table is astronomically large. This is our first concrete encounter with the boundary between tractable and intractable problems, a theme we will return to in the complexity unit.

4 Unbounded Knapsack: Coin Change and Rod Cutting

In the *unbounded* knapsack variant, each item type may be used *any number of times*. Two classic instances appear in this week’s exercises.

4.1 Coin change: minimum number of coins

Definition 4.1 (Coin Change – minimization). *Given coin denominations $c_1, \dots, c_k$ (unlimited supply of each) and an amount A , find the minimum number of coins summing to exactly A , or report that A cannot be made.*

Let $dp[a]$ be the minimum number of coins summing to a :

$$dp[0] = 0, \quad dp[a] = 1 + \min_{c_i \leq a} dp[a - c_i] \quad (\text{or } +\infty \text{ if no } c_i \leq a \text{ helps}).$$

The answer is $dp[A]$, or -1 if it is $+\infty$. The table has $A+1$ entries, each requiring an $O(k)$ minimization, so the running time is $O(kA)$ – again pseudo-polynomial. Note the crucial difference from Subset Sum: because coins are *reusable*, we iterate a in *ascending* order and do *not* reverse the loop. Unlike the Week 4 greedy coin-change algorithm (correct only for “canonical” coin systems), this DP is correct for *any* coin system. Problem 8 implements it.

4.2 Coin change: number of ways

Definition 4.2 (Coin Change – counting). *With the same input, count the number of distinct combinations (multisets, where order does not matter) of coins summing to A .*

Let $ways[a]$ be the number of combinations summing to a , with $ways[0] = 1$ (the empty combination). To count *combinations* rather than *ordered sequences*, we loop over the coins on the *outside* and amounts on the *inside*:

for c in coins: **for** $a = c$ to A : $ways[a] += ways[a - c]$.

Putting the coin loop outside fixes a canonical order in which coins are “committed,” so each combination is counted exactly once. Swapping the loop order would instead count permutations. Problem 9 implements this and is the same idea behind LeetCode’s *Coin Change II*.

4.3 Rod cutting

Definition 4.3 (Rod Cutting). *Given a rod of integer length n and a price table where p_ℓ is the price of a piece of length ℓ , cut the rod into integer pieces to maximize total revenue.*

This is unbounded knapsack with “item ℓ ” = a piece of length ℓ , weight ℓ , value p_ℓ , and capacity n (each length usable repeatedly). Let $rev[L]$ be the maximum revenue from a rod of length L :

$$rev[0] = 0, \quad rev[L] = \max_{1 \leq \ell \leq L} (p_\ell + rev[L - \ell]).$$

The running time is $O(n^2)$. Recording, for each L , the cut length ℓ achieving the maximum lets us reconstruct the list of pieces. Problem 10 implements both the revenue and the cut list.

5 Longest Increasing Subsequence

As a final example of a one-dimensional DP with a slightly different flavor:

Definition 5.1 (Longest Increasing Subsequence (LIS)). *Given a sequence $a_1, \dots, a_n$, find the length of the longest subsequence (not necessarily contiguous, but keeping the original order) that is strictly increasing.*

Let $L[i]$ be the length of the longest increasing subsequence *ending at index i* :

$$L[i] = 1 + \max(\{L[j] : j < i, a_j < a_i\} \cup \{0\}).$$

The answer is $\max_i L[i]$ (or 0 if the sequence is empty). Each $L[i]$ scans all earlier indices, giving $O(n^2)$. (An $O(n \log n)$ patience-sorting algorithm also exists; we defer it.) Problem 11 implements the $O(n^2)$ DP and verifies it against a brute-force search over all subsequences on small inputs – a good template for empirically validating any DP.

6 Summary and Looking Ahead

This week introduced **dynamic programming** as a systematic technique for problems with *optimal substructure* and *overlapping subproblems*, contrasting *top-down memoization* with *bottom-up tabulation*. The case studies were:

- **Weighted Interval Scheduling:** the recurrence $M[j] = \max(M[j - 1], w_j + M[p(j)])$, with $p(j)$ found by binary search, solved in $O(n \log n)$, plus solution reconstruction by traceback.

- **Subset Sum and 0/1 Knapsack:** the 2-D table $dp[i][w]$ in $O(nW)$ time, with item reconstruction – and the observation that $O(nW)$ is only *pseudo-polynomial*, since W is exponential in the input’s bit-length (Knapsack is NP-hard).
- **Unbounded knapsack:** coin change (minimum coins, and counting combinations) and rod cutting, where items may be reused – distinguished from 0/1 by the loop direction/order.
- **Longest Increasing Subsequence:** an $O(n^2)$ DP, verified against brute force.

The recurring discipline is the five-step recipe of Section 1.4: define the subproblem, write the recurrence (centered on a single decision), state base cases, evaluate efficiently, and reconstruct the optimal solution – not just its value.

Next week (Dynamic Programming II, Kleinberg & Tardos Ch. 6 continued) we extend this toolkit to *two-dimensional* alignment problems – **sequence alignment / edit distance**, the basis of spell-checkers and computational biology – and to **shortest-path** problems on graphs with negative edges via the Bellman–Ford and Floyd–Warshall dynamic programs, where the subproblems are indexed by “number of edges allowed” or “set of intermediate vertices.”